МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных технологий

Отчет по практической работе №3

по дисциплине «Тестирование и верификация ПО»

Тема практической работы: «Подходы для разработки программного обеспечения»

Выполнил:
Студент группы ИКБО-71-24 Клочков Ф.А.

Проверил: Ассистент кафедры МОСИТ,
Петрова А.А

Москва 2026

Содержание

1 Цель и задачи работы

2 Теоретические основы

2.1 Test-Driven Development (TDD)

2.2 Acceptance Test-Driven Development (ATDD)

2.3 Behavior-Driven Development (BDD)

2.4 Specification by Example (SDD)

3 Практическая часть

3.1 Вариант задания и условия разработки

3.2 Архитектура программного продукта

3.3 Разработка с использованием TDD

3.4 Разработка с использованием ATDD

3.5 Разработка с использованием BDD

3.6 Разработка с использованием SDD

3.7 Итоговое тестирование

4 Анализ подходов

Заключение

Список использованных источников

Приложение А. Исходный код программного продукта

Приложение Б. Модульные, приёмочные и спецификационные тесты

Приложение В. Сценарии BDD и определения шагов

Приложение Г. Протокол итогового прогона тестов

1 Цель и задачи работы

Цель работы – изучить и применить подходы к разработке программного обеспечения, основанные на тестировании (TDD, ATDD, BDD и SDD), для повышения качества, надёжности и поддерживаемости кода.

В ходе работы выполнены следующие задачи:

1) изучены теоретические основы методологий TDD, ATDD, BDD и SDD (Specification by Example);

2) исследованы преимущества и недостатки каждого подхода;

3) по варианту 18 разработан программный продукт «Музыкальный плеер», причём каждая часть его функциональности создана с применением одного из четырёх подходов;

4) проанализировано влияние интеграции тестирования на архитектуру и качество продукта;

5) подготовлен отчёт с выводами и рекомендациями по интеграции подходов в процесс разработки.

2 Теоретические основы

2.1 Test-Driven Development (TDD)

TDD (разработка через тестирование) – методология, при которой тест на новую функциональность пишется до её реализации. Разработка ведётся короткими циклами «красный – зелёный – рефакторинг»: сначала создаётся тест, описывающий ожидаемое поведение, и запускается – он падает, поскольку кода ещё нет; затем пишется минимальный код, достаточный для прохождения теста; после этого код улучшается без изменения поведения, а тесты подтверждают, что ничего не сломано.

Основной инструмент TDD – модульные (unit) тесты, проверяющие отдельные функции и классы в изоляции. Подход ориентирован на разработчика: он задаёт техническую спецификацию кода и даёт быструю обратную связь.

2.2 Acceptance Test-Driven Development (ATDD)

ATDD (разработка через приёмочные тесты) начинается с совместного обсуждения требований заказчиком, аналитиком, тестировщиком и разработчиком. Результатом обсуждения становятся приёмочные критерии – сценарии вида «предусловие – действия – ожидаемый результат», которые автоматизируются до начала реализации. Функциональность считается готовой, когда проходят все приёмочные тесты.

В отличие от TDD, приёмочные тесты проверяют систему «снаружи» – через пользовательский или программный интерфейс, – и отвечают на вопрос «делает ли система то, что нужно заказчику».

2.3 Behavior-Driven Development (BDD)

BDD (разработка через поведение) развивает идеи TDD: поведение системы описывается на естественном языке в виде сценариев со структурой Given – When – Then (Дано – Когда – Тогда). Сценарии записываются на языке Gherkin и одновременно служат документацией и автоматизированными тестами: каждому шагу сценария сопоставляется функция-определение шага (step definition) в коде.

Для автоматизации используются Cucumber (Java, JavaScript), SpecFlow (.NET) и behave (Python). Gherkin поддерживает русские ключевые слова (Функция, Сценарий, Дано, Когда, Тогда), что делает сценарии понятными участникам проекта, не владеющим программированием.

2.4 Specification by Example (SDD)

Specification by Example (в методических указаниях – SDD) – подход, при котором требования описываются не абстрактными формулировками, а конкретными примерами: таблицами входных данных и ожидаемых результатов. Примеры собираются вместе с заказчиком, устраняют двусмысленность требований и напрямую преобразуются в автоматизированные тесты.

Ключевое свойство подхода – «живая документация»: если тесты

читают примеры непосредственно из спецификации, то спецификация не может разойтись с поведением системы.

3 Практическая часть

3.1 Вариант задания и условия разработки

Номер студенческого билета – 24И0418, последние две цифры – 18, что соответствует варианту 18 «Музыкальный плеер». Функции по заданию: воспроизведение, пауза, переключение треков. Для подходов заданы: TDD – тесты корректного переключения треков и управления воспроизведением; ATDD – сценарии воспроизведения для проверки пользовательского опыта; BDD – сценарий «Given треки загружены, When нажимаю «play», Then начинается воспроизведение»; SDD – спецификации с входными данными (список треков) и ожидаемыми состояниями плеера.

Чтобы каждый подход действительно управлял разработкой, а не проверял уже готовый код, функциональность разделена между этапами: каждый этап начинается с тестов на ещё не реализованное поведение (таблица 2). Условия разработки приведены в таблице 1.

Таблица 1 – Условия разработки

Параметр	Значение
Язык программирования	Python 3.14.7
Модульные, приёмочные и спецификационные тесты	pytest 9.2.0, pytest-cov 7.1.0
Программный интерфейс (ATDD)	FastAPI 0.162.0, тестовый клиент на httpx 0.28.1
Сценарии поведения (BDD)	behave 1.3.3, язык Gherkin (русские ключевые слова)
Сервер для ручной проверки API	uvicorn 0.54.0
Контроль версий	git: каждый этап «красный / зелёный / рефакторинг» – отдельный коммит
Операционная система	Fedora Linux 43

Таблица 2 – Функциональность плеера и подход, которым она разработана

Функция	Описание	Подход
Трек	название, исполнитель, длительность; пустое название и неположительная длительность запрещены	TDD
Загрузка плейлиста	первый трек становится текущим, плеер остановлен	TDD
play / pause / stop	запуск, приостановка и продолжение, остановка; пауза без воспроизведения – ошибка	TDD
next / previous	переключение треков; на последнем треке next останавливает плеер	TDD
REST API	управление плеером по HTTP, коды ответов 200 / 400 / 409	ATDD
Режим повтора	после последнего трека – первый, перед первым – последний	BDD
Ход времени (tick)	позиция в треке, автопереход к следующему треку с переносом остатка	SDD
Правило «назад»	если трек играет более 3 с, previous перезапускает его	SDD

3.2 Архитектура программного продукта

Продукт разделён на доменный слой и слой интерфейса. Доменная логика (модуль player/core.py) не зависит от фреймворков и содержит классы Track (неизменяемый трек), Player (состояние плеера, плейлист, позиция) и перечисление PlayerState. REST API (player/api.py) – тонкая обёртка, которая преобразует HTTP-запросы в вызовы методов Player, а исключения предметной области – в коды ответов. Связь модулей с тестовыми наборами показана на рисунке 1, состояния плеера – на рисунке 2.

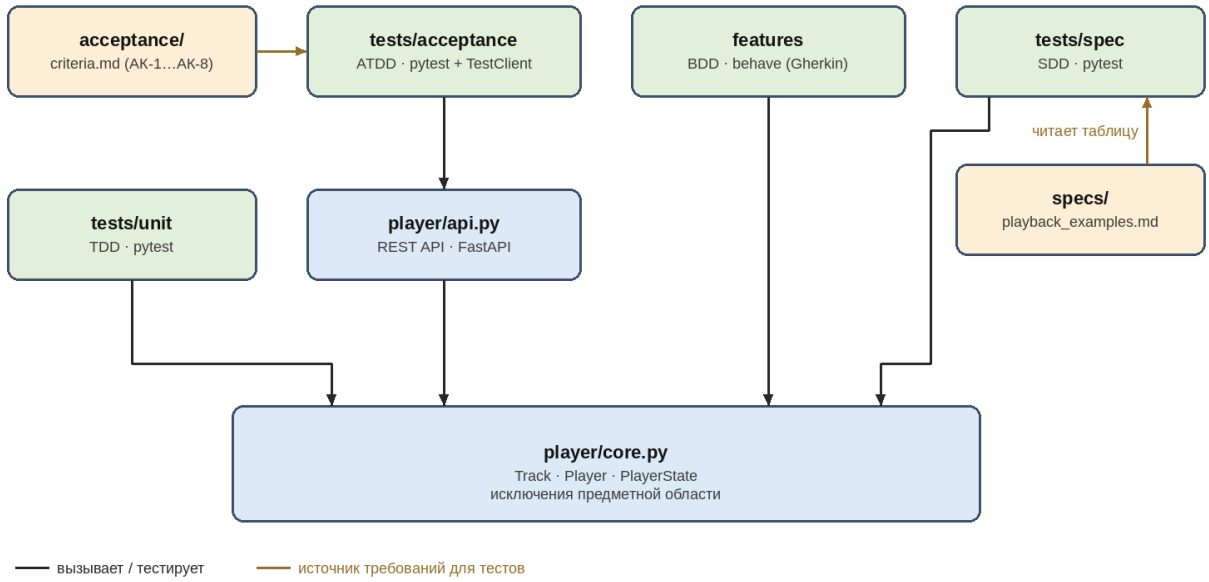


Рисунок 1 – Структура проекта и связь кода с тестами



Рисунок 2 – Диаграмма состояний плеера

Таблица 3 – Состав проекта

Путь	Назначение
player/core.py	доменная логика плеера
player/api.py	REST API на FastAPI
tests/unit/test_player.py	модульные тесты (TDD)
acceptance/criteria.md	приёмочные критерии AK-1 – AK-8 (ATDD)
tests/acceptance/test_player_api.py	автоматизированные приёмочные тесты (ATDD)
features/*.feature	сценарии на языке Gherkin (BDD)
features/steps/player_steps.py	определения шагов behave (BDD)
specs/playback_examples.md	спецификация на примерах (SDD)
tests/spec/test_playback_spec.py	тесты, генерируемые из таблицы спецификации (SDD)
logs/	протоколы прогонов каждого этапа

3.3 Разработка с использованием TDD

С помощью TDD разработано ядро плеера: трек, загрузка плейлиста, воспроизведение, пауза, остановка и переключение треков.

Шаг 1. Создание тестов

До написания кода составлены 25 модульных тестов (с учётом параметризации), описывающих ожидаемое поведение. Тесты сгруппированы по функциям (таблица 4); общие тестовые данные вынесены в фикстуры pytest. Фрагмент тестов приведён в листинге 1, полный текст – в приложении Б.

Таблица 4 – Модульные тесты ядра плеера

Группа	Тесты	Что проверяется
Трек	3 (5 с параметрами)	сохранение метаданных, запрет пустого названия и неположительной длительности
Загрузка	4	новый плеер пуст и остановлен; первый трек текущий; повторная загрузка сбрасывает состояние; плейлист копируется
play / pause / stop	6 (7)	запуск, ошибка при пустом плейлисте, повторный play, пауза, ошибка паузы вне воспроизведения, продолжение после паузы, остановка
next / previous	7 (8)	переход вперёд и назад, сохранение воспроизведения, остановка при смене трека на паузе и в конце плейлиста, ошибки при пустом плейлисте

Листинг 1 – Фрагмент модульных тестов переключения треков

```
001  @pytest.fixture
002  def player(tracks):
003      p = Player()
004      p.load(tracks)
005      return p
006
007  def test_next_keeps_playing(player, tracks):
008      player.play()
009      player.next()
010      assert player.state is PlayerState.PLAYING
011      assert player.current_track == tracks[1]
012
013  def test_next_on_last_track_stops_playback(player, tracks):
014      player.play()
015      player.next()
016      player.next()
017      player.next()
018      assert player.current_track == tracks[2]
019      assert player.state is PlayerState.STOPPED
```

Шаг 2. Запуск тестов

Для того чтобы тесты собирались, создан модуль-заглушка с объявлениями классов и исключений, конструкторы которых выбрасывают NotImplementedError. Все 25 тестов не проходят: 9 падают в теле теста, 16 – при подготовке фикстуры (рисунок 3). Это ожидаемый «красный» этап.

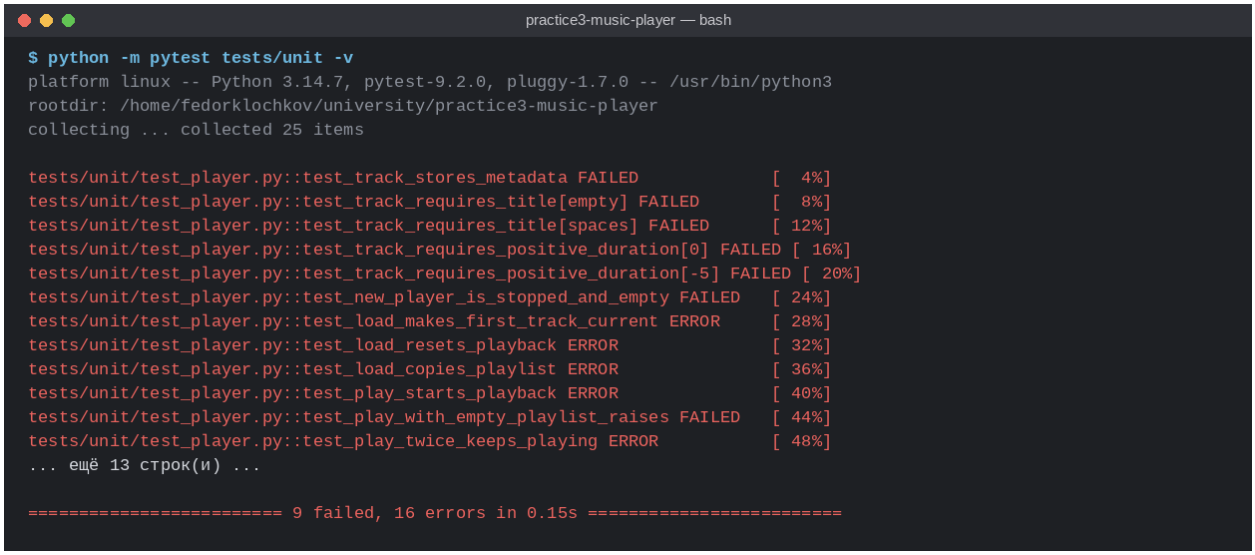


Рисунок 3 – Красный этап TDD: тесты падают, реализация отсутствует

Шаг 3. Реализация функциональности

Написан минимальный код, достаточный для прохождения тестов. На этом этапе допускались дублирование (проверка пустого плейлиста повторена в трёх методах) и открытые для записи атрибуты. Методы переключения треков в первой версии приведены в листинге 2.

Листинг 2 – Минимальная реализация переключения треков (этап «зелёный»)

```
001  def next(self) -> None:
002      self._require_playlist()
003      if self._index == len(self._playlist) - 1:
004          self._state = PlayerState.STOPPED
005      else:
006          self._switch_to(self._index + 1)
007
008  def previous(self) -> None:
009      self._require_playlist()
010      self._switch_to(max(self._index - 1, 0))
```

Шаг 4. Повторный запуск тестов

Все 25 тестов пройдены (рисунок 4).

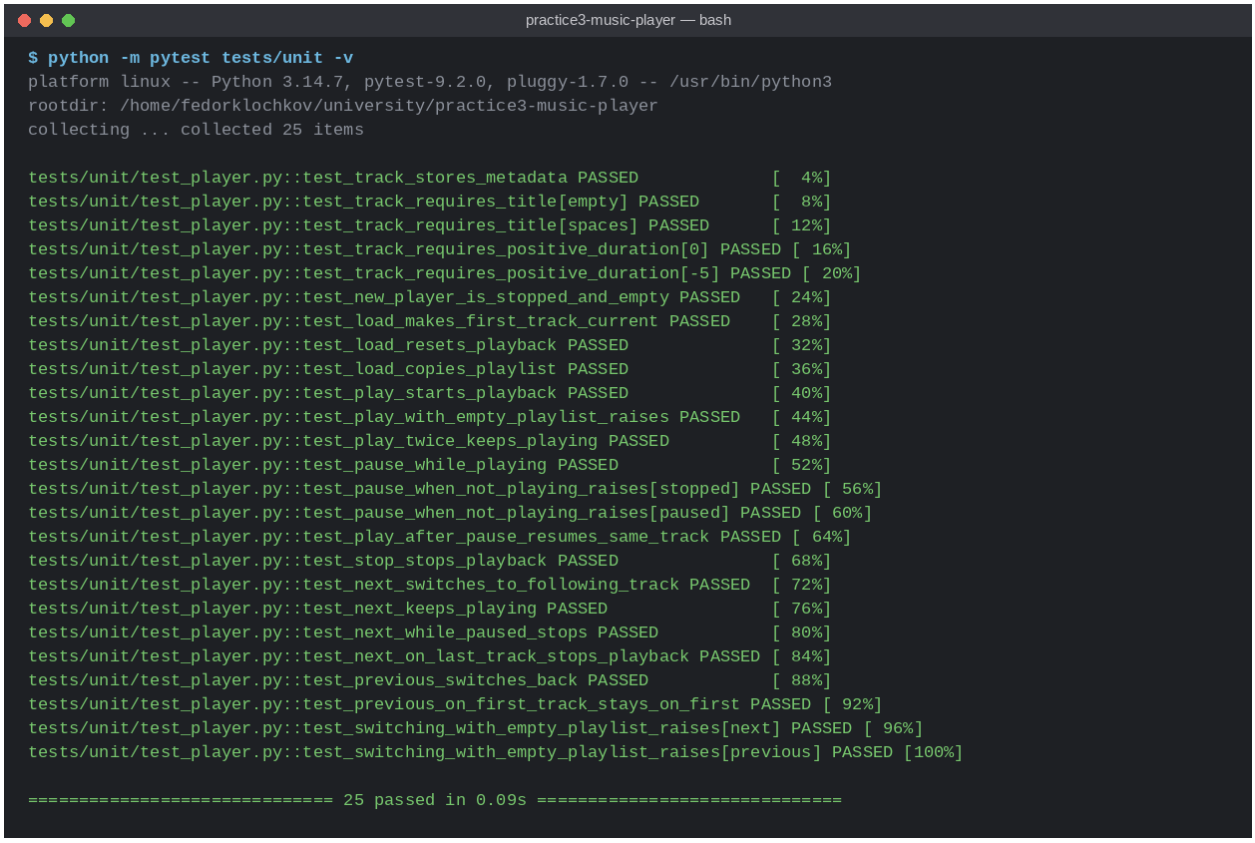


Рисунок 4 – Зелёный этап TDD: все модульные тесты пройдены

Шаг 5. Рефакторинг

После прохождения тестов выполнен рефакторинг без изменения поведения:

1) Track преобразован в неизменяемый dataclass с проверкой данных в __post_init__;

2) состояние плеера закрыто от внешней записи: доступ только через свойства playlist, state, index, current_track;

3) повторяющаяся проверка пустого плейлиста вынесена в метод _require_playlist;

4) правило «смена трека сохраняет воспроизведение, иначе плеер останавливается» вынесено в метод _switch_to;

5) добавлены аннотации типов и документирующие строки исключений.

Листинг 3 – Переключение треков после рефакторинга

```
001  def next(self) -> None:
002      self._require_playlist()
003      if self._index == len(self._playlist) - 1:
004          self._state = PlayerState.STOPPED
005      else:
006          self._switch_to(self._index + 1)
007
008  def previous(self) -> None:
009      self._require_playlist()
010      self._switch_to(max(self._index - 1, 0))
011
012  def _require_playlist(self) -> None:
013      if not self._playlist:
014          raise EmptyPlaylistError("Плейлист пуст")
015
016  def _switch_to(self, index: int) -> None:
017      """Переключение сохраняет воспроизведение, иначе плеер
останавливается."""
018      self._index = index
019      if self._state is not PlayerState.PLAYING:
020          self._state = PlayerState.STOPPED
```

Повторный прогон подтвердил, что поведение не изменилось; покрытие операторов модуля модульными тестами составило 98 % (рисунок 5). Непокрытым осталось только свойство index, которое используется на следующем этапе в REST API.

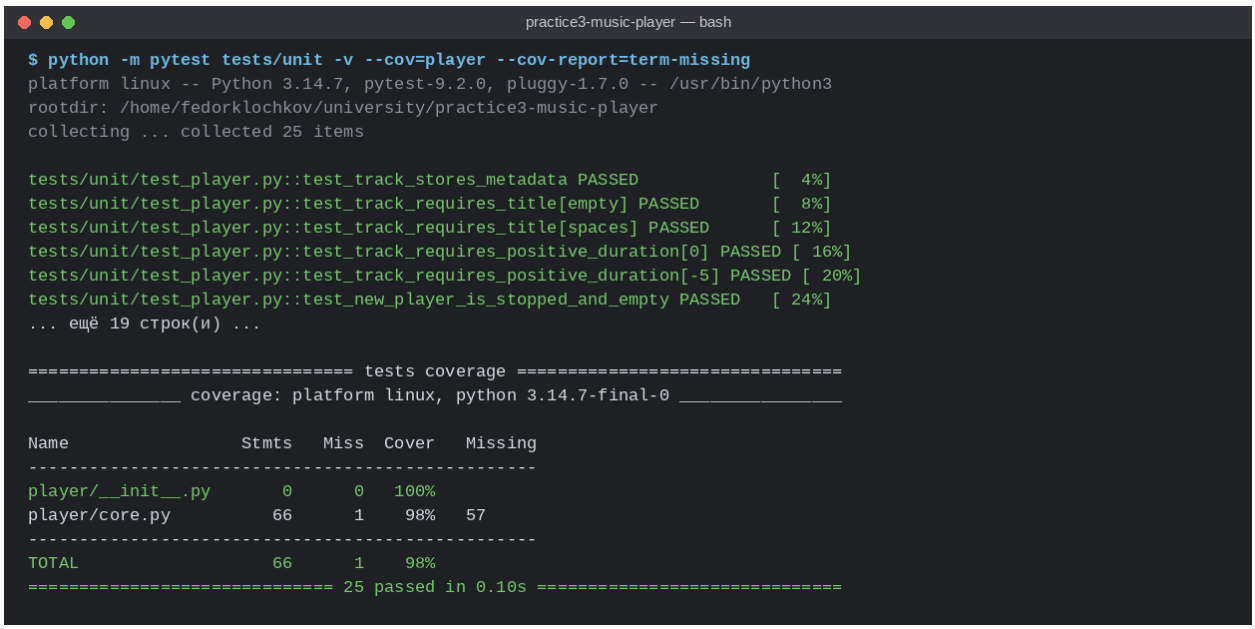


Рисунок 5 – Прогон после рефакторинга с измерением покрытия

3.4 Разработка с использованием ATDD

С помощью ATDD разработан программный интерфейс плеера (REST

API), через который с плеером работает пользовательское приложение.

Шаг 1. Согласование приёмочных критериев

Работа индивидуальная, поэтому роли заказчика (владельца продукта), тестировщика и разработчика выполнял автор. Требования сформулированы с точки зрения пользователя и зафиксированы в файле acceptance/criteria.md до начала реализации (таблица 5). Тестовый плейлист: «Intro» (180 с), «Song» (200 с), «Outro» (150 с).

Таблица 5 – Приёмочные критерии REST API плеера

№	Сценарий	Предусловие	Действия	Ожидаемый результат
АК-1	Загрузка плейлиста	Плеер пуст	Пользователь загружает 3 трека	Ответ 200, в плейлисте 3 трека, текущий трек «Intro», плеер остановлен
АК-2	Запуск воспроизведения	Плейлист загружен	Пользователь нажимает «Play»	Ответ 200, состояние «playing», звучит «Intro»
АК-3	Пауза и продолжение	Воспроизводится «Intro»	«Pause», затем «Play»	После паузы состояние «paused»; после повторного «Play» снова «playing», трек тот же
АК-4	Переключение треков	Воспроизводится «Intro»	«Next», затем «Previous»	После «Next» звучит «Song», после «Previous» — снова «Intro», воспроизведение не прерывается
АК-5	Конец плейлиста	Воспроизводится «Outro» (последний)	«Next»	Плеер остановлен, текущим остаётся «Outro»
АК-6	Пустой плейлист	Плейлист не загружен	«Play»	Ответ 400 с сообщением «Плейлист пуст»
АК-7	Пауза без воспроизведения	Плейлист загружен, плеер остановлен	«Pause»	Ответ 409 с сообщением о недопустимой операции, состояние не меняется
АК-8	Некорректный трек	Загружен корректный плейлист	Загрузка трека с длительностью 0	Ответ 400, прежний плейлист сохраняется

Шаг 2. Автоматизация приёмочных тестов

Каждый критерий преобразован в тест, который обращается к приложению через HTTP с помощью тестового клиента FastAPI (TestClient), не запуская сервер. Тесты проверяют коды ответов и содержимое JSON, то

есть поведение, наблюдаемое пользователем (листинг 4).

Листинг 4 – Приёмочные тесты АК-5 и АК-6

```
001  def press(client, button):
002      return client.post(f"/player/{button}")
003
004  def test_ac5_end_of_playlist_stops(loaded):
005      press(loaded, "play")
006      press(loaded, "next")
007      press(loaded, "next")
008
009      status = press(loaded, "next").json()
010
011      assert status["state"] == "stopped"
012      assert status["track"]["title"] == "Outro"
013
014  def test_ac6_play_with_empty_playlist(client):
015      response = press(client, "play")
016
017      assert response.status_code == 400
018      assert response.json()["detail"] == "Плейлист пуст"
```

Шаг 3. Запуск тестов

Создано пустое приложение FastAPI без маршрутов. Все 8 приёмочных тестов не проходят: на любой запрос сервер отвечает 404 (рисунок 6).

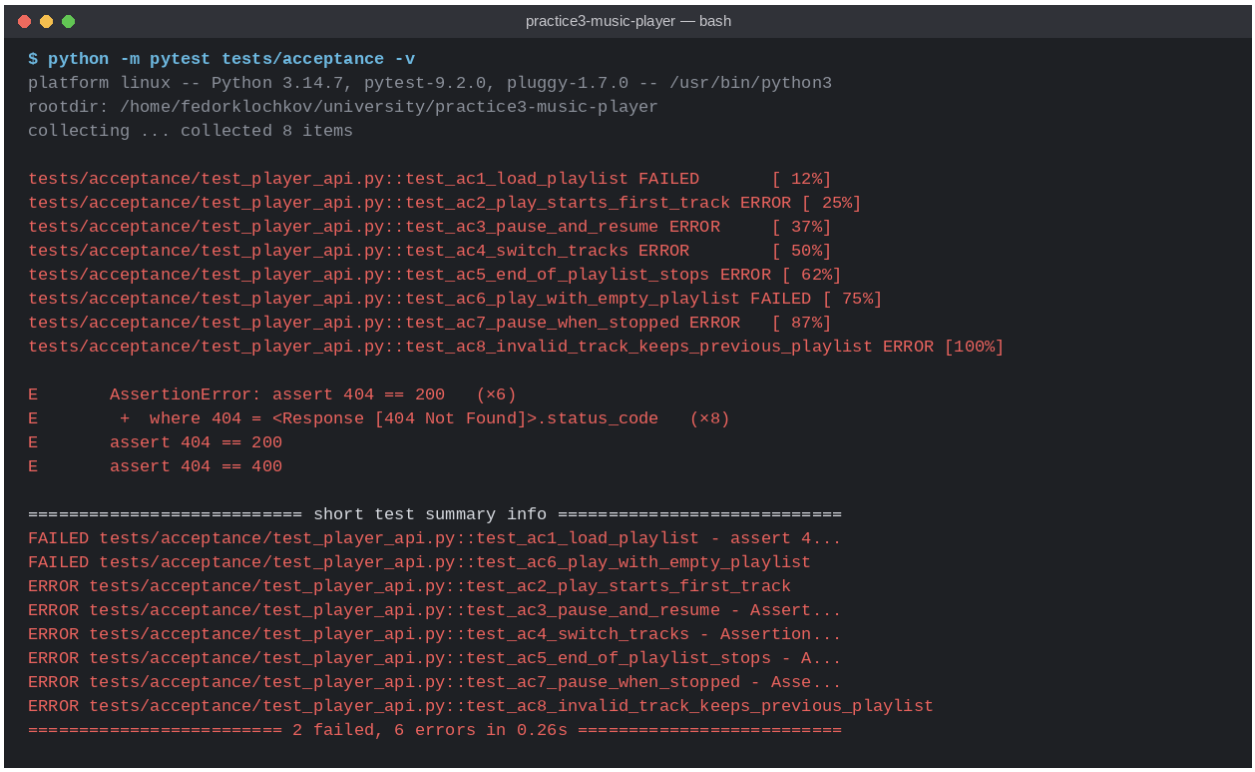


Рисунок 6 – Красный этап ATDD: маршруты API отсутствуют

Шаг 4. Реализация функциональности

Реализован REST API (таблица 6). Исключения предметной области централизованно преобразуются в коды ответов: пустой плейлист и некорректный трек – 400, недопустимая операция в текущем состоянии – 409. Новый плейлист сначала полностью проверяется и только затем загружается, поэтому некорректный запрос не портит прежний плейлист (критерий АК-8). Фрагмент кода – в листинге 5.

Таблица 6 – Методы REST API

Метод и путь	Назначение	Ответы
GET /player	текущее состояние: режим, трек, позиция, плейлист	200
PUT /playlist	загрузка плейлиста	200, 400
POST /player/play	запуск или продолжение воспроизведения	200, 400
POST /player/pause	приостановка	200, 409
POST /player/stop	остановка	200
POST /player/next	следующий трек	200, 400
POST /player/previous	предыдущий трек	200, 400

Листинг 5 – Фабрика приложения REST API

```
001  def create_app(player: Player | None = None) -> FastAPI:
002      app = FastAPI(title="Музыкальный плеер")
003      player = player or Player()
004
005      for error, code in ERROR_CODES.items():
006          app.add_exception_handler(error, _error_handler(code))
007
008      @app.get("/player")
009      def get_status() -> dict:
010          return _status(player)
011
012      @app.put("/playlist")
013      def load_playlist(body: PlaylistIn) -> dict:
014          player.load([Track(t.title, t.artist, t.duration) for t in
body.tracks])
015          return _status(player)
016
017      actions = {
018          "play": player.play,
019          "pause": player.pause,
020          "stop": player.stop,
021          "next": player.next,
022          "previous": player.previous,
023      }
024      for name, action in actions.items():
025          app.post(f"/player/{name}", name=name)(_action_endpoint(player,
action))
026
027      return app
```

Шаг 5. Повторный запуск тестов

Все 8 приёмочных тестов пройдены (рисунок 7), модульные тесты при этом также остались зелёными. Рефакторинг не потребовался: обработчики ошибок и действий с самого начала вынесены в функции-фабрики. Работа API дополнительно проверена вручную через uvicorn и curl.

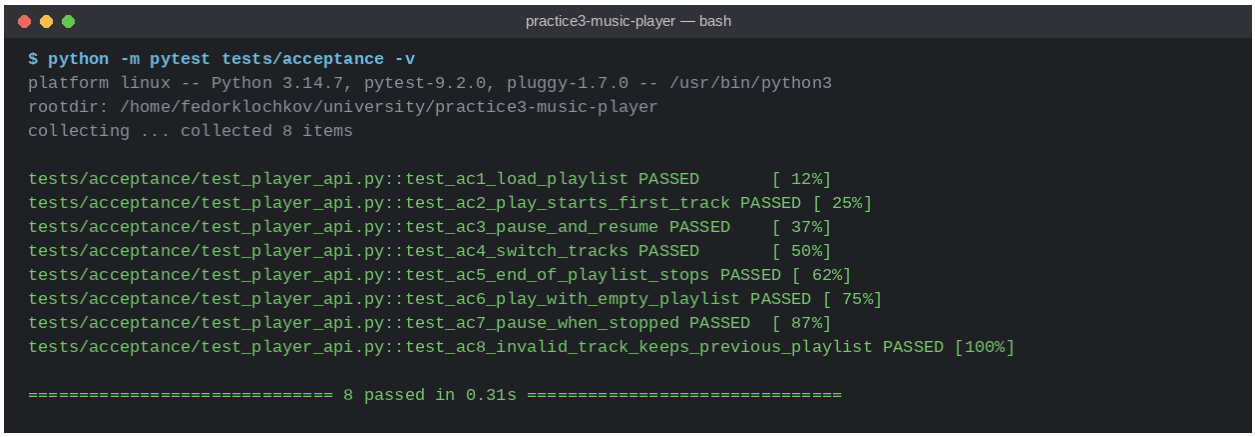


Рисунок 7 – Зелёный этап ATDD: все приёмочные критерии выполнены

3.5 Разработка с использованием BDD

С помощью BDD описано поведение плеера с точки зрения слушателя и разработана новая функция – режим повтора плейлиста.

Шаг 1. Написание сценариев

Сценарии записаны на языке Gherkin с русскими ключевыми словами (директива # language: ru). Предыстория загружает тестовый плейлист в виде таблицы; сценарий «Запуск воспроизведения кнопкой «play»» дословно реализует сценарий варианта «Given треки загружены, When нажимаю «play», Then начинается воспроизведение». Режим повтора описан структурами сценария (Scenario Outline) с таблицами примеров: одни и те же шаги выполняются при выключенном и включённом повторе. Всего 8 сценариев в двух файлах; фрагмент – в листинге 6, полный текст – в приложении В.

Листинг 6 – Фрагмент файла features/playback.feature

```
# language: ru
Функция: Управление воспроизведением музыкального плеера
  Чтобы слушать музыку в удобном мне порядке
  Как пользователь музыкального плеера
  Я хочу запускать, приостанавливать и переключать треки

  Предыстория:
    Дано треки загружены в плеер:
      | название | исполнитель | длительность |
      | Intro    | Band        | 180          |
      | Song     | Band        | 200          |
      | Outro    | Band        | 150          |

  Сценарий: Запуск воспроизведения кнопкой «play»
    Когда я нажимаю "play"
    Тогда начинается воспроизведение
    И звучит трек "Intro"
  ...
  Структура сценария: Переключение вперёд с последнего трека
    Дано плеер воспроизводит трек "Outro"
    И режим повтора <повтор>
    Когда я нажимаю "next"
    Тогда текущий трек "<трек>"
    И плеер в состоянии "<состояние>"

    Примеры:
      | повтор   | трек  | состояние       |
      | выключен | Outro | остановлен      |
      | включён  | Intro | воспроизведение |
```

Шаг 2. Автоматизация сценариев

При первом запуске behave сообщил, что все 46 шагов не определены, и предложил заготовки функций (рисунок 8).

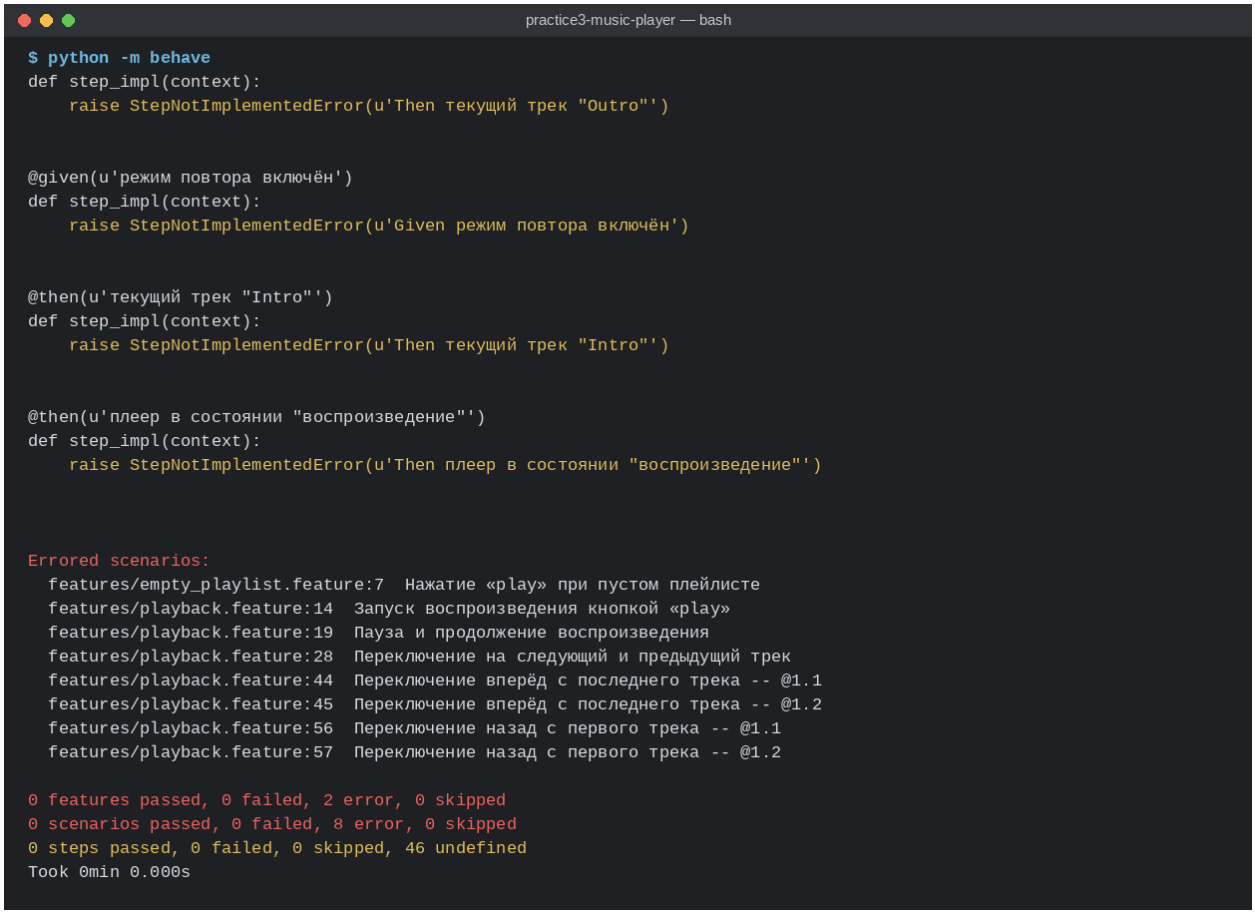


Рисунок 8 – Первый запуск behave: шаги не определены

Затем написаны определения шагов (листинг 7). Шаги параметризованы: например, шаблон «я нажимаю "{button}"» обрабатывает все кнопки плеера, а словари переводят русские названия состояний и режимов в значения перечислений. Перед каждым сценарием в environment.py создаётся новый экземпляр плеера, поэтому сценарии не зависят друг от друга.

Листинг 7 – Фрагмент определений шагов

```
001  @given("треки загружены в плеер:")
002  def step_load_tracks(context):
003      context.player.load(
004          Track(row["название"], row["исполнитель"],
int(row["длительность"]))
005          for row in context.table
006      )
007
008  @given("режим повтора {mode}")
009  def step_repeat_mode(context, mode):
010      context.player.repeat = REPEAT[mode]
011
012  @step('я нажимаю "{button}"')
013  def step_press(context, button):
014      try:
015          getattr(context.player, button)()
016      except (EmptyPlaylistError, PlayerStateError) as exc:
017          context.error = exc
018
019  @then('текущий трек "{title}"')
020  def step_current_track(context, title):
021      assert context.player.current_track.title == title,
context.player.current_track
```

Повторный запуск: 6 сценариев проходят на уже существующей логике, 2 сценария режима повтора падают – после последнего трека плеер останавливается вместо перехода к первому (рисунок 9).

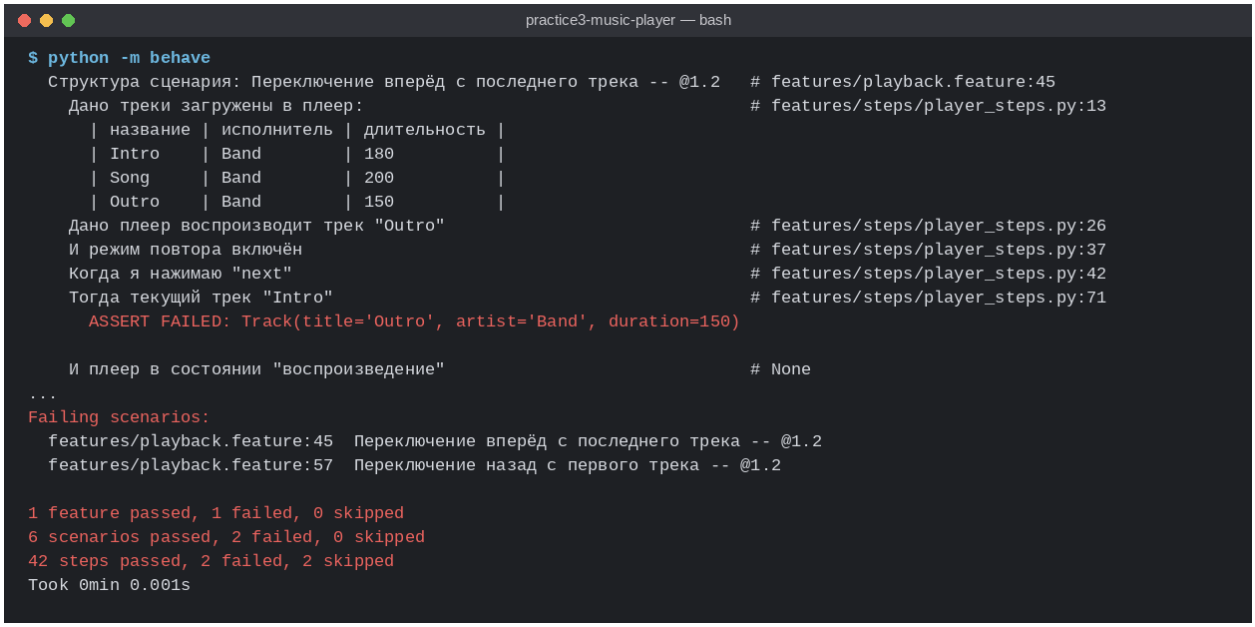


Рисунок 9 – Красный этап BDD: сценарии режима повтора не выполняются

Шаг 3. Реализация поведения

В класс Player добавлен атрибут repeat, а методы next и previous

дополнены ветвями для режима повтора (листинг 8).

Листинг 8 – Переключение треков с учётом режима повтора

```
001  def next(self) -> None:
002      self._require_playlist()
003      if self._index < len(self._playlist) - 1:
004          self._switch_to(self._index + 1)
005      elif self.repeat:
006          self._switch_to(0)
007      else:
008          self._state = PlayerState.STOPPED
009
010  def previous(self) -> None:
011      self._require_playlist()
012      if self._index > 0:
013          self._switch_to(self._index - 1)
014      elif self.repeat:
015          self._switch_to(len(self._playlist) - 1)
016      else:
017          self._switch_to(0)
```

Шаг 4. Повторный запуск сценариев

Все 8 сценариев (46 шагов) выполнены успешно (рисунок 10), модульные и приёмочные тесты остались зелёными.

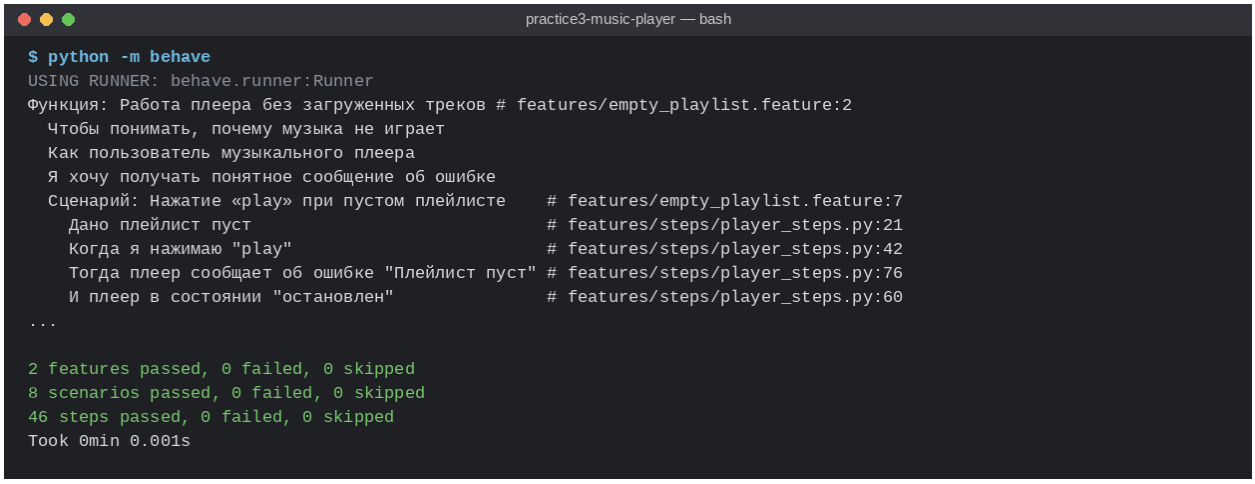


Рисунок 10 – Зелёный этап BDD: все сценарии выполнены

Шаг 5. Рефакторинг

Изменение затронуло два метода и не создало дублирования, поэтому отдельный рефакторинг не потребовался; сценарии остаются исполняемой документацией поведения плеера.

3.6 Разработка с использованием SDD

С помощью Specification by Example разработаны ход времени

воспроизведения, автоматический переход к следующему треку и правило кнопки «назад» – поведение, которое удобнее всего задать набором конкретных примеров.

Шаг 1. Документирование примеров

Примеры собраны в файле specs/playback_examples.md. Сначала сформулированы правила:

1) `tick N` — прошло N секунд. Позиция растёт только во время воспроизведения;

2) Когда позиция достигает длительности трека, плеер переходит к следующему; «лишние» секунды переносятся на следующий трек;

3) После последнего трека плеер останавливается на нём с позицией 0; при включённом повторе продолжает с первого трека;

4) «previous» при позиции больше 3 секунд перезапускает текущий трек, иначе переключает на предыдущий;

5) Любое переключение трека и «stop» сбрасывают позицию в 0, пауза её сохраняет;

6) Отрицательное время недопустимо.

Затем для стандартного плейлиста «Intro» (180 с), «Song» (200 с), «Outro» (150 с) и пустого плейлиста составлены 17 примеров: обычные случаи, граничные значения (ровно длительность трека, ровно 3 с для правила «назад», конец плейлиста) и ошибочные действия (таблица 7). Действия перечисляются через точку с запятой, «tick N» означает, что прошло N секунд.

Таблица 7 – Спецификация на примерах: состояние плеера после действий

№	Плейлист	Повтор	Действия	Трек	Состояние	Позиция	Ошибка
1	стандартный	нет	play	Intro	playing	0	—
2	стандартный	нет	play; tick 60	Intro	playing	60	—
3	стандартный	нет	play; tick 60; pause; tick 30	Intro	paused	60	—
4	стандартный	нет	play; tick 60; pause; play	Intro	playing	60	—
5	стандартный	нет	tick 60	Intro	stopped	0	—
6	стандартный	нет	play; tick 180	Song	playing	0	—
7	стандартный	нет	play; tick 190	Song	playing	10	—
8	стандартный	нет	play; tick 530	Outro	stopped	0	—

№	Плейлист	Повтор	Действия	Трек	Состояние	Позиция	Ошибка
9	стандартный	да	play; tick 540	Intro	playing	10	—
10	стандартный	нет	play; tick 60; next	Song	playing	0	—
11	стандартный	нет	play; tick 60; previous	Intro	playing	0	—
12	стандартный	нет	play; next; tick 3; previous	Intro	playing	0	—
13	стандартный	нет	play; next; tick 4; previous	Song	playing	0	—
14	стандартный	нет	play; tick 60; stop	Intro	stopped	0	—
15	стандартный	нет	pause	Intro	stopped	0	PlayerStateError
16	стандартный	нет	play; tick -5	Intro	playing	0	ValueError
17	пустой	нет	play	—	stopped	0	EmptyPlaylistError

Шаг 2. Создание спецификаций и автоматизированных тестов

Тест не дублирует данные таблицы, а читает её прямо из файла спецификации: функция read_examples разбирает Markdown-таблицу, а декоратор pytest.mark.parametrize превращает каждую строку в отдельный тест (листинг 9). Добавление строки в спецификацию автоматически добавляет тест – спецификация становится «живой документацией».

Листинг 9 – Генерация тестов из таблицы спецификации

```
001  def read_examples(path: Path) -> list[dict[str, str]]:
002      """Строки таблицы из раздела «Примеры» в виде словарей «колонка → значение»."""
003      lines = path.read_text(encoding="utf-8").split("## Примеры", 1)[1].splitlines()
004      rows = [
005          [cell.strip() for cell in line.strip().strip("|").split("|")]
006          for line in lines
007          if line.startswith("|")
008      ]
009      header, _separator, *body = rows
010      return [dict(zip(header, row)) for row in body]
011
012  def apply(player: Player, action: str) -> None:
013      name, *args = action.split()
014      getattr(player, name)(*map(int, args))
015
016  @pytest.mark.parametrize("ex", EXAMPLES, ids=[f"N{ex['№']}: {ex['Действия']}" for ex in EXAMPLES])
017  def test_playback_example(ex):
018      player = Player()
019      player.load(PLAYLISTS[ex["Плейлист"]])
020      player.repeat = ex["Повтор"] == "да"
021      *setup, last = [a.strip() for a in ex["Действия"].split(";")]
022
```

```
023      for action in setup:
024          apply(player, action)
025      if ex["Ошибка"] == NONE:
026          apply(player, last)
027      else:
028          with pytest.raises(ERRORS[ex["Ошибка"]]):
029              apply(player, last)
030
031      title = player.current_track.title if player.current_track else NONE
032      assert title == ex["Трек"]
033      assert player.state.value == ex["Состояние"]
034      assert player.position == int(ex["Позиция"])
```

Шаг 3. Запуск тестов

Все 17 примеров не проходят: у плеера ещё нет метода tick и свойства position (рисунок 11).

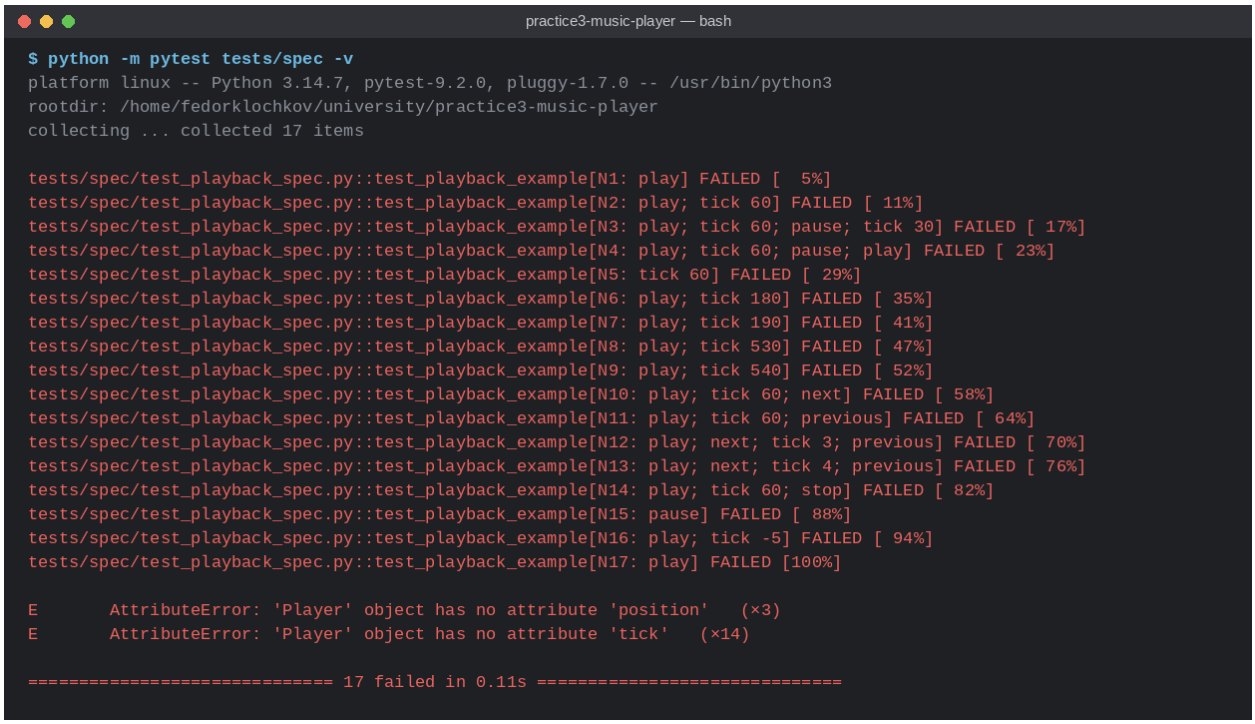


Рисунок 11 – Красный этап SDD: функциональность времени отсутствует

Шаг 4. Реализация функциональности

В Player добавлены позиция воспроизведения, метод tick и порог перезапуска трека RESTART_THRESHOLD = 3 с. Метод tick в цикле переносит «лишние» секунды на следующие треки, поэтому один вызов может пройти несколько треков (примеры 8 и 9). Переключение трека и остановка сбрасывают позицию, пауза сохраняет её (листинг 10). Позиция и режим повтора добавлены также в ответ REST API.

Листинг 10 – Ход времени и правило кнопки «назад»

```
001  def tick(self, seconds: int) -> None:
002      """Продвигает воспроизведение на заданное число секунд."""
003      if seconds < 0:
004          raise ValueError("Время не может быть отрицательным")
005      if self._state is not PlayerState.PLAYING:
006          return
007      self._position += seconds
008      while self._state is PlayerState.PLAYING and self._position >=
self.current_track.duration:
009          overflow = self._position - self.current_track.duration
010          self.next()
011          if self._state is PlayerState.PLAYING:
012              self._position = overflow
013
014  def previous(self) -> None:
015      self._require_playlist()
016      if self._position > RESTART_THRESHOLD:
017          self._switch_to(self._index)
018      elif self._index > 0:
019          self._switch_to(self._index - 1)
020      elif self.repeat:
021          self._switch_to(len(self._playlist) - 1)
022      else:
023          self._switch_to(0)
```

Шаг 5. Повторный запуск тестов

Все 17 примеров спецификации выполнены (рисунок 12). Регрессий в тестах предыдущих этапов нет.

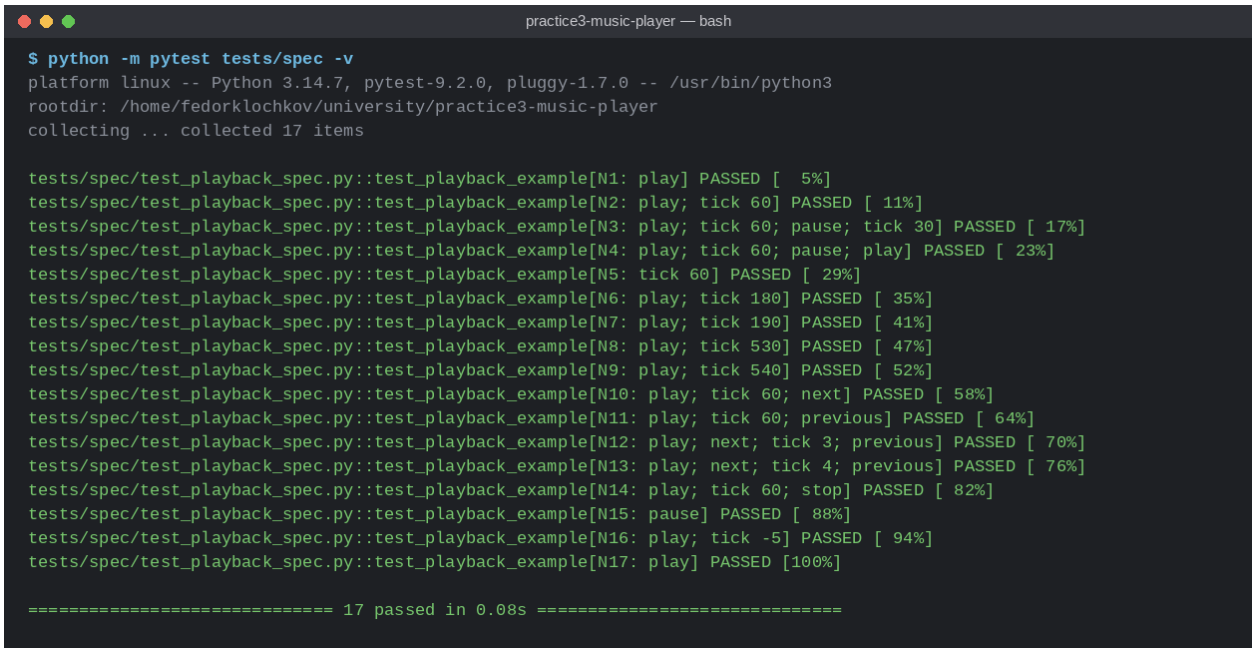


Рисунок 12 – Зелёный этап SDD: все примеры спецификации выполнены

3.7 Итоговое тестирование

После завершения всех этапов выполнен общий прогон тестов (рисунок 13, полный протокол – в приложении Г). Сводные результаты приведены в таблице 8. Покрытие операторов кода тестами pytest составило 99 %; две непокрытые строки – возврат None для пустого трека в API и ветвь previous в режиме повтора, которая покрыта сценариями behave.

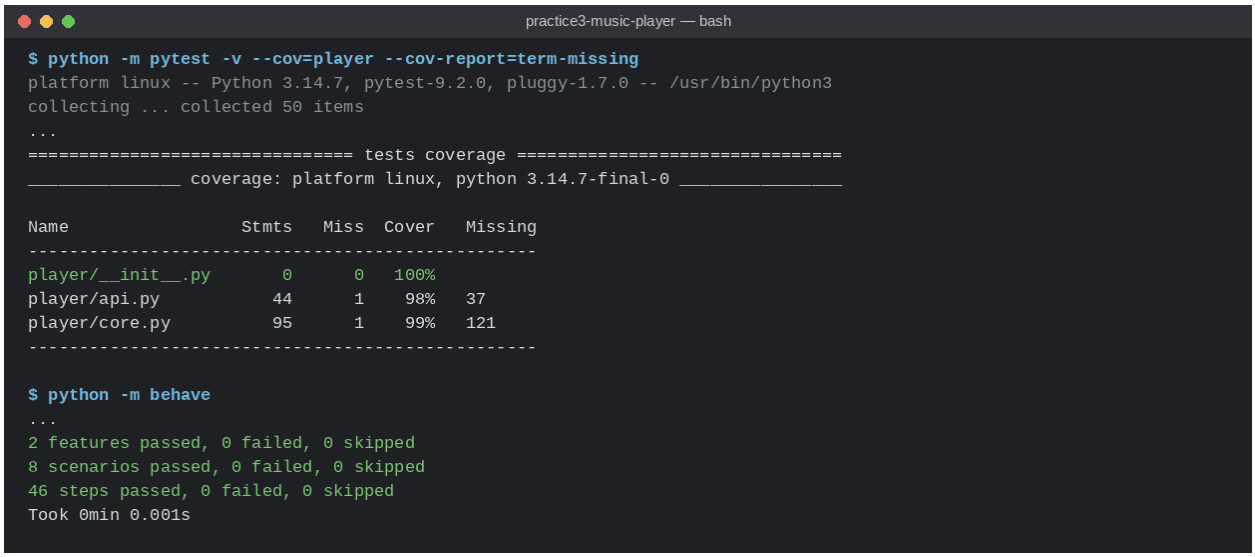


Рисунок 13 – Итоговый прогон всех тестов

Таблица 8 – Результаты итогового тестирования

Подход	Набор тестов	Количество	Результат
TDD	tests/unit	25 тестов	все пройдены
ATDD	tests/acceptance	8 тестов	все пройдены
BDD	features	8 сценариев, 46 шагов	все выполнены
SDD	tests/spec	17 примеров	все пройдены
Итого		58 проверок	покрытие 99 %

История разработки сохранена в git: каждый «красный», «зелёный» этап и рефакторинг оформлены отдельным коммитом, что подтверждает порядок работы «тест – код» (таблица 9). Протоколы каждого прогона сохранены в каталоге logs.

Таблица 9 – История коммитов

№	Сообщение коммита
1	TDD (red): модульные тесты и заглушка
2	TDD (green): минимальная реализация плеера

№	Сообщение коммита
3	TDD (refactor): неизменяемый Track, инкапсуляция состояния, устранение дублирования
4	ATDD (red): приёмочные критерии и тесты REST API
5	ATDD (green): REST API плеера на FastAPI
6	BDD (red): сценарии Gherkin, шаги не определены
7	BDD (red): определения шагов, сценарии режима повтора падают
8	BDD (green): режим повтора плейлиста
9	SDD (red): спецификация на примерах и тесты по таблице
10	SDD (green): ход времени, автопереход, правило перезапуска трека
11	Позиция и режим повтора в ответе API, README, итоговый прогон

4 Анализ подходов

Все четыре подхода интегрируют тестирование в разработку, но различаются тем, кто формулирует проверки, на каком уровне они работают и кому адресован результат (таблица 10).

Таблица 10 – Сравнение подходов

Критерий	TDD	ATDD	BDD	SDD
Основа	модульные тесты	приёмочные критерии	сценарии поведения	таблицы примеров
Кто формулирует	разработчик	заказчик, аналитик, тестировщик, разработчик	команда и бизнес	заказчик и команда
Уровень проверки	класс, функция	система через интерфейс	поведение системы	бизнес-правило
Язык описания	код	текст критериев и код	Gherkin (естественный язык)	таблицы
Инструменты в работе	pytest	pytest, FastAPI TestClient	behave	pytest (parametrize)
Что разработано	ядро плеера	REST API	режим повтора	ход времени, правило «назад»

Таблица 11 – Преимущества и недостатки подходов

Подход	Преимущества	Недостатки
TDD	быстрая обратная связь; высокое покрытие; безопасный рефакторинг; тесты заставляют проектировать слабосвязанный код	не проверяет соответствие требованиям заказчика; тесты нужно поддерживать при изменении интерфейсов; требует дисциплины

Подход	Преимущества	Недостатки
ATDD	общее понимание требований до разработки; критерии готовности; проверка системы целиком	нужны встречи всех сторон; тесты медленнее модульных; при ошибке сложнее найти её место
BDD	сценарии понятны не программистам; документация не устаревает; переиспользуемые шаги	дополнительный слой (Gherkin и шаги); риск громоздких сценариев; поддержка определений шагов
SDD	устраняет неоднозначность; удобно задаёт граничные случаи; новый пример – новый тест	сложное поведение трудно уложить в таблицу; разбор таблиц требует кода; таблицы разрастаются

Влияние на архитектуру. Требование тестируемости заметно повлияло на структуру продукта. Чтобы модульные тесты были быстрыми и независимыми, логика плеера отделена от HTTP и не использует системное время: время передаётся явно через tick, поэтому примеры SDD детерминированы. Приёмочные тесты потребовали фабрики create_app, создающей изолированный экземпляр приложения для каждого теста. Иерархия исключений предметной области позволила централизованно сопоставить ошибки с кодами HTTP. Рефакторинг на этапе TDD, выполненный под защитой тестов, закрыл состояние плеера от внешнего изменения и устранил дублирование.

Влияние на качество. Тесты каждого следующего этапа запускались вместе с предыдущими, поэтому добавление режима повтора и хода времени не нарушило уже реализованное поведение – регрессии обнаруживались бы сразу. Граничные случаи (ровно длительность трека, ровно 3 с, конец плейлиста, отрицательное время) зафиксированы явно и проверяются автоматически.

Рекомендации по интеграции подходов. Подходы не исключают, а дополняют друг друга, и их удобно применять как уровни одной пирамиды тестирования:

1) ATDD – на уровне пользовательской истории: приёмочные критерии как «определение готовности» задачи;

2) BDD – для описания поведения, которое важно обсуждать с бизнесом; сценарии Gherkin служат живой документацией;

3) SDD – для бизнес-правил с большим числом вариантов (расчёты, тарифы, переходы состояний) в виде таблиц;

4) TDD – на уровне реализации каждого класса и функции, обеспечивая быструю обратную связь и безопасный рефакторинг;

5) все наборы тестов запускаются автоматически при каждом изменении (непрерывная интеграция).

Заключение

В ходе работы изучены подходы TDD, ATDD, BDD и SDD и с их помощью разработан программный продукт «Музыкальный плеер» (вариант 18). Каждая часть функциональности создана по циклу «тест – падение – реализация – успешный прогон – рефакторинг»: ядро плеера – через модульные тесты (TDD), REST API – через приёмочные критерии (ATDD), режим повтора – через сценарии на Gherkin (BDD), ход времени и правило кнопки «назад» – через таблицу примеров (SDD).

Оценка качества программного продукта: все 58 автоматизированных проверок выполняются успешно, покрытие операторов составляет 99 %, ошибочные действия пользователя обрабатываются с понятными сообщениями и корректными кодами ответа. Документация продукта – приёмочные критерии, сценарии Gherkin и спецификация на примерах – исполняема: она проверяется при каждом прогоне и не может разойтись с кодом.

Сделан вывод, что TDD обеспечивает надёжность реализации, ATDD и BDD – соответствие ожиданиям пользователя и общий язык с заказчиком, SDD – однозначность бизнес-правил. Наибольший эффект даёт их совместное применение на разных уровнях разработки.

Список использованных источников

1. Практическая работа. Подходы для разработки программного обеспечения : методические указания. – М. : РТУ МИРЭА, 2026.

2. ГОСТ 19.401-78. Единая система программной документации. Текст программы. Требования к содержанию и оформлению.

3. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы.

4. Бек К. Экстремальное программирование: разработка через тестирование. – СПб. : Питер, 2003. – 224 с.

5. Аджич Г. Specification by Example: How Successful Teams Deliver the Right Software. – Manning, 2011. – 296 p.

6. pytest documentation [Электронный ресурс]. – URL: https://docs.pytest.org/ (дата обращения: 31.10.2026).

7. behave documentation [Электронный ресурс]. – URL: https://behave.readthedocs.io/ (дата обращения: 31.10.2026).

8. Gherkin Reference [Электронный ресурс]. – URL: https://cucumber.io/docs/gherkin/reference/ (дата обращения: 31.10.2026).

9. FastAPI. Testing [Электронный ресурс]. – URL: https://fastapi.tiangolo.com/tutorial/testing/ (дата обращения: 31.10.2026).

Приложение А. Исходный код программного продукта

Ниже приведён итоговый текст модулей. Нумерация строк соответствует файлам проекта.

Листинг А.1 – Модуль player/core.py

```
001  """Доменная логика музыкального плеера."""
002
003  from __future__ import annotations
004
005  from dataclasses import dataclass
006  from enum import Enum
007  from typing import Iterable
008
009  RESTART_THRESHOLD = 3
010  """Если прошло больше секунд, «previous» перезапускает текущий трек."""
011
012
013  class InvalidTrackError(ValueError):
014      """Некорректные метаданные трека."""
015
016
017  class EmptyPlaylistError(Exception):
018      """Операция требует непустого плейлиста."""
019
020
021  class PlayerStateError(Exception):
022      """Операция недопустима в текущем состоянии плеера."""
023
024
025  class PlayerState(Enum):
026      STOPPED = "stopped"
027      PLAYING = "playing"
028      PAUSED = "paused"
029
030
031  @dataclass(frozen=True)
032  class Track:
033      title: str
034      artist: str
035      duration: int
036
037      def __post_init__(self) -> None:
038          if not self.title.strip():
039              raise InvalidTrackError("Название трека не может быть
пустым")
040          if self.duration <= 0:
041              raise InvalidTrackError("Длительность трека должна быть
положительной")
042
043
044  class Player:
045      def __init__(self) -> None:
046          self._playlist: list[Track] = []
047          self._index = 0
048          self._state = PlayerState.STOPPED
049          self._position = 0
050          self.repeat = False
051
```

```
052      @property
053      def playlist(self) -> list[Track]:
054          return list(self._playlist)
055
056      @property
057      def state(self) -> PlayerState:
058          return self._state
059
060      @property
061      def index(self) -> int:
062          return self._index
063
064      @property
065      def position(self) -> int:
066          """Позиция воспроизведения в текущем треке, секунды."""
067          return self._position
068
069      @property
070      def current_track(self) -> Track | None:
071          return self._playlist[self._index] if self._playlist else None
072
073      def load(self, tracks: Iterable[Track]) -> None:
074          self._playlist = list(tracks)
075          self._index = 0
076          self._state = PlayerState.STOPPED
077          self._position = 0
078
079      def play(self) -> None:
080          self._require_playlist()
081          self._state = PlayerState.PLAYING
082
083      def pause(self) -> None:
084          if self._state is not PlayerState.PLAYING:
085              raise PlayerStateError("Пауза возможна только во время
воспроизведения")
086          self._state = PlayerState.PAUSED
087
088      def stop(self) -> None:
089          self._state = PlayerState.STOPPED
090          self._position = 0
091
092      def tick(self, seconds: int) -> None:
093          """Продвигает воспроизведение на заданное число секунд."""
094          if seconds < 0:
095              raise ValueError("Время не может быть отрицательным")
096          if self._state is not PlayerState.PLAYING:
097              return
098          self._position += seconds
099          while self._state is PlayerState.PLAYING and self._position >=
self.current_track.duration:
100              overflow = self._position - self.current_track.duration
101              self.next()
102              if self._state is PlayerState.PLAYING:
103                  self._position = overflow
104
105      def next(self) -> None:
106          self._require_playlist()
107          if self._index < len(self._playlist) - 1:
108              self._switch_to(self._index + 1)
109          elif self.repeat:
110              self._switch_to(0)
111          else:
112              self.stop()
113
```

```
    def previous(self) -> None:
        self._require_playlist()
        if self._position > RESTART_THRESHOLD:
            self._switch_to(self._index)
        elif self._index > 0:
            self._switch_to(self._index - 1)
        elif self.repeat:
            self._switch_to(len(self._playlist) - 1)
        else:
            self._switch_to(0)

    def _require_playlist(self) -> None:
        if not self._playlist:
            raise EmptyPlaylistError("Плейлист пуст")

    def _switch_to(self, index: int) -> None:
        """Переключение сохраняет воспроизведение, иначе плеер
останавливается."""
        self._index = index
        self._position = 0
        if self._state is not PlayerState.PLAYING:
            self._state = PlayerState.STOPPED
```

Листинг А.2 – Модуль player/api.py

```
"""REST API музыкального плеера — тонкий слой поверх доменной логики."""

from __future__ import annotations

from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from pydantic import BaseModel

from player.core import (
    EmptyPlaylistError,
    InvalidTrackError,
    Player,
    PlayerStateError,
    Track,
)


class TrackIn(BaseModel):
    title: str
    artist: str
    duration: int


class PlaylistIn(BaseModel):
    tracks: list[TrackIn]


ERROR_CODES = {
    EmptyPlaylistError: 400,
    InvalidTrackError: 400,
    PlayerStateError: 409,
}


def _track_to_dict(track: Track | None) -> dict | None:
    if track is None:
```

```
037          return None
038      return {"title": track.title, "artist": track.artist, "duration":
track.duration}
039
040
041  def _status(player: Player) -> dict:
042      return {
043          "state": player.state.value,
044          "index": player.index,
045          "position": player.position,
046          "repeat": player.repeat,
047          "track": _track_to_dict(player.current_track),
048          "playlist": [_track_to_dict(t) for t in player.playlist],
049      }
050
051
052  def create_app(player: Player | None = None) -> FastAPI:
053      app = FastAPI(title="Музыкальный плеер")
054      player = player or Player()
055
056      for error, code in ERROR_CODES.items():
057          app.add_exception_handler(error, _error_handler(code))
058
059      @app.get("/player")
060      def get_status() -> dict:
061          return _status(player)
062
063      @app.put("/playlist")
064      def load_playlist(body: PlaylistIn) -> dict:
065          player.load([Track(t.title, t.artist, t.duration) for t in
body.tracks])
066          return _status(player)
067
068      actions = {
069          "play": player.play,
070          "pause": player.pause,
071          "stop": player.stop,
072          "next": player.next,
073          "previous": player.previous,
074      }
075      for name, action in actions.items():
076          app.post(f"/player/{name}", name=name)(_action_endpoint(player,
action))
077
078      return app
079
080
081  def _error_handler(code: int):
082      async def handle(_: Request, exc: Exception) -> JSONResponse:
083          return JSONResponse(status_code=code, content={"detail":
str(exc)})
084
085      return handle
086
087
088  def _action_endpoint(player: Player, action):
089      def endpoint() -> dict:
090          action()
091          return _status(player)
092
093      return endpoint
094
095
096  app = create_app()
```

Для сравнения с итоговой версией приведена первая реализация ядра, полученная на «зелёном» этапе TDD до рефакторинга.

Листинг А.3 – Модуль player/core.py после этапа «зелёный» TDD

```
001  """Доменная логика музыкального плеера."""
002
003  from __future__ import annotations
004
005  from dataclasses import dataclass
006  from enum import Enum
007  from typing import Iterable
008
009
010  class InvalidTrackError(ValueError):
011      """Некорректные метаданные трека."""
012
013
014  class EmptyPlaylistError(Exception):
015      """Операция требует непустого плейлиста."""
016
017
018  class PlayerStateError(Exception):
019      """Операция недопустима в текущем состоянии плеера."""
020
021
022  class PlayerState(Enum):
023      STOPPED = "stopped"
024      PLAYING = "playing"
025      PAUSED = "paused"
026
027
028  @dataclass(frozen=True)
029  class Track:
030      title: str
031      artist: str
032      duration: int
033
034      def __post_init__(self) -> None:
035          if not self.title.strip():
036              raise InvalidTrackError("Название трека не может быть
пустым")
037          if self.duration <= 0:
038              raise InvalidTrackError("Длительность трека должна быть
положительной")
039
040
041  class Player:
042      def __init__(self) -> None:
043          self._playlist: list[Track] = []
044          self._index = 0
045          self._state = PlayerState.STOPPED
046
047      @property
048      def playlist(self) -> list[Track]:
049          return list(self._playlist)
050
051      @property
052      def state(self) -> PlayerState:
```

```
053          return self._state
054
055      @property
056      def index(self) -> int:
057          return self._index
058
059      @property
060      def current_track(self) -> Track | None:
061          return self._playlist[self._index] if self._playlist else None
062
063      def load(self, tracks: Iterable[Track]) -> None:
064          self._playlist = list(tracks)
065          self._index = 0
066          self._state = PlayerState.STOPPED
067
068      def play(self) -> None:
069          self._require_playlist()
070          self._state = PlayerState.PLAYING
071
072      def pause(self) -> None:
073          if self._state is not PlayerState.PLAYING:
074              raise PlayerStateError("Пауза возможна только во время
воспроизведения")
075          self._state = PlayerState.PAUSED
076
077      def stop(self) -> None:
078          self._state = PlayerState.STOPPED
079
080      def next(self) -> None:
081          self._require_playlist()
082          if self._index == len(self._playlist) - 1:
083              self._state = PlayerState.STOPPED
084          else:
085              self._switch_to(self._index + 1)
086
087      def previous(self) -> None:
088          self._require_playlist()
089          self._switch_to(max(self._index - 1, 0))
090
091      def _require_playlist(self) -> None:
092          if not self._playlist:
093              raise EmptyPlaylistError("Плейлист пуст")
094
095      def _switch_to(self, index: int) -> None:
096          """Переключение сохраняет воспроизведение, иначе плеер
останавливается."""
097          self._index = index
098          if self._state is not PlayerState.PLAYING:
099              self._state = PlayerState.STOPPED
```

Приложение Б. Модульные, приёмочные и спецификационные тесты

Листинг Б.1 – Модульные тесты tests/unit/test_player.py (TDD)

```
001  """TDD: модульные тесты доменной логики плеера."""
002
003  import pytest
004
005  from player.core import (
006      EmptyPlaylistError,
007      InvalidTrackError,
008      Player,
009      PlayerState,
010      PlayerStateError,
011      Track,
012  )
013
014
015  @pytest.fixture
016  def tracks():
017      return [
018          Track("Intro", "Band", 180),
019          Track("Song", "Band", 200),
020          Track("Outro", "Band", 150),
021      ]
022
023
024  @pytest.fixture
025  def player(tracks):
026      p = Player()
027      p.load(tracks)
028      return p
029
030
031  # --- Трек
------------------------------------------------------------
032
033  def test_track_stores_metadata():
034      track = Track("Intro", "Band", 180)
035      assert (track.title, track.artist, track.duration) == ("Intro",
"Band", 180)
036
037
038  @pytest.mark.parametrize("title", ["", "   "], ids=["empty", "spaces"])
039  def test_track_requires_title(title):
040      with pytest.raises(InvalidTrackError):
041          Track(title, "Band", 180)
042
043
044  @pytest.mark.parametrize("duration", [0, -5])
045  def test_track_requires_positive_duration(duration):
046      with pytest.raises(InvalidTrackError):
047          Track("Intro", "Band", duration)
048
049
050  # --- Загрузка плейлиста
-----------------------------------------------
051
052  def test_new_player_is_stopped_and_empty():
053      p = Player()
054      assert p.state is PlayerState.STOPPED
055      assert p.current_track is None
```

```
    assert p.playlist == []


def test_load_makes_first_track_current(player, tracks):
    assert player.playlist == tracks
    assert player.current_track == tracks[0]
    assert player.state is PlayerState.STOPPED


def test_load_resets_playback(player, tracks):
    player.play()
    player.next()
    player.load(tracks[::-1])
    assert player.state is PlayerState.STOPPED
    assert player.current_track == tracks[2]


def test_load_copies_playlist(tracks):
    p = Player()
    p.load(tracks)
    tracks.clear()
    assert len(p.playlist) == 3


# --- Воспроизведение и пауза
-----------------------------------------
def test_play_starts_playback(player, tracks):
    player.play()
    assert player.state is PlayerState.PLAYING
    assert player.current_track == tracks[0]


def test_play_with_empty_playlist_raises():
    with pytest.raises(EmptyPlaylistError):
        Player().play()


def test_play_twice_keeps_playing(player):
    player.play()
    player.play()
    assert player.state is PlayerState.PLAYING


def test_pause_while_playing(player):
    player.play()
    player.pause()
    assert player.state is PlayerState.PAUSED


@pytest.mark.parametrize(
    "prepare",
    [lambda p: None, lambda p: (p.play(), p.pause())],
    ids=["stopped", "paused"],
)
def test_pause_when_not_playing_raises(player, prepare):
    prepare(player)
    with pytest.raises(PlayerStateError):
        player.pause()


def test_play_after_pause_resumes_same_track(player, tracks):
    player.play()
    player.next()
```

```
    player.pause()
    player.play()
    assert player.state is PlayerState.PLAYING
    assert player.current_track == tracks[1]


def test_stop_stops_playback(player):
    player.play()
    player.stop()
    assert player.state is PlayerState.STOPPED


# --- Переключение треков
-----------------------------------------------

def test_next_switches_to_following_track(player, tracks):
    player.next()
    assert player.current_track == tracks[1]


def test_next_keeps_playing(player, tracks):
    player.play()
    player.next()
    assert player.state is PlayerState.PLAYING
    assert player.current_track == tracks[1]


def test_next_while_paused_stops(player, tracks):
    player.play()
    player.pause()
    player.next()
    assert player.state is PlayerState.STOPPED
    assert player.current_track == tracks[1]


def test_next_on_last_track_stops_playback(player, tracks):
    player.play()
    player.next()
    player.next()
    player.next()
    assert player.current_track == tracks[2]
    assert player.state is PlayerState.STOPPED


def test_previous_switches_back(player, tracks):
    player.play()
    player.next()
    player.previous()
    assert player.current_track == tracks[0]
    assert player.state is PlayerState.PLAYING


def test_previous_on_first_track_stays_on_first(player, tracks):
    player.play()
    player.previous()
    assert player.current_track == tracks[0]
    assert player.state is PlayerState.PLAYING


@pytest.mark.parametrize("action", ["next", "previous"])
def test_switching_with_empty_playlist_raises(action):
    with pytest.raises(EmptyPlaylistError):
        getattr(Player(), action)()
```

Листинг Б.2 – Приёмочные тесты tests/acceptance/test_player_api.py (ATDD)

```
001  """ATDD: приёмочные тесты REST API плеера (критерии —
acceptance/criteria.md)."""
002
003  import pytest
004  from fastapi.testclient import TestClient
005
006  from player.api import create_app
007
008  PLAYLIST = {
009      "tracks": [
010          {"title": "Intro", "artist": "Band", "duration": 180},
011          {"title": "Song", "artist": "Band", "duration": 200},
012          {"title": "Outro", "artist": "Band", "duration": 150},
013      ]
014  }
015
016
017  @pytest.fixture
018  def client():
019      return TestClient(create_app())
020
021
022  @pytest.fixture
023  def loaded(client):
024      assert client.put("/playlist", json=PLAYLIST).status_code == 200
025      return client
026
027
028  def press(client, button):
029      return client.post(f"/player/{button}")
030
031
032  def test_ac1_load_playlist(client):
033      response = client.put("/playlist", json=PLAYLIST)
034
035      assert response.status_code == 200
036      body = response.json()
037      assert len(body["playlist"]) == 3
038      assert body["track"]["title"] == "Intro"
039      assert body["state"] == "stopped"
040
041
042  def test_ac2_play_starts_first_track(loaded):
043      response = press(loaded, "play")
044
045      assert response.status_code == 200
046      assert response.json()["state"] == "playing"
047      assert response.json()["track"]["title"] == "Intro"
048
049
050  def test_ac3_pause_and_resume(loaded):
051      press(loaded, "play")
052
053      paused = press(loaded, "pause").json()
054      assert paused["state"] == "paused"
055
056      resumed = press(loaded, "play").json()
057      assert resumed["state"] == "playing"
```

```
058      assert resumed["track"]["title"] == "Intro"
059
060
061  def test_ac4_switch_tracks(loaded):
062      press(loaded, "play")
063
064      after_next = press(loaded, "next").json()
065      assert (after_next["track"]["title"], after_next["state"]) ==
("Song", "playing")
066
067      after_previous = press(loaded, "previous").json()
068      assert (after_previous["track"]["title"], after_previous["state"])
== ("Intro", "playing")
069
070
071  def test_ac5_end_of_playlist_stops(loaded):
072      press(loaded, "play")
073      press(loaded, "next")
074      press(loaded, "next")
075
076      status = press(loaded, "next").json()
077
078      assert status["state"] == "stopped"
079      assert status["track"]["title"] == "Outro"
080
081
082  def test_ac6_play_with_empty_playlist(client):
083      response = press(client, "play")
084
085      assert response.status_code == 400
086      assert response.json()["detail"] == "Плейлист пуст"
087
088
089  def test_ac7_pause_when_stopped(loaded):
090      response = press(loaded, "pause")
091
092      assert response.status_code == 409
093      assert "Пауза возможна только" in response.json()["detail"]
094      assert loaded.get("/player").json()["state"] == "stopped"
095
096
097  def test_ac8_invalid_track_keeps_previous_playlist(loaded):
098      bad = {"tracks": [{"title": "Broken", "artist": "Band", "duration":
0}]}
099
100      response = loaded.put("/playlist", json=bad)
101
102      assert response.status_code == 400
103      status = loaded.get("/player").json()
104      assert [t["title"] for t in status["playlist"]] == ["Intro", "Song",
"Outro"]
```

Листинг Б.3 – Тесты спецификации tests/spec/test_playback_spec.py (SDD)

```
001  """SDD: тесты, построенные по таблице примеров
specs/playback_examples.md."""
002
003  from pathlib import Path
004
005  import pytest
006
```

```
from player.core import EmptyPlaylistError, Player, PlayerStateError,
Track

SPEC = Path(__file__).resolve().parents[2] / "specs" /
"playback_examples.md"
PLAYLISTS = {
    "стандартный": [Track("Intro", "Band", 180), Track("Song", "Band",
200), Track("Outro", "Band", 150)],
    "пустой": [],
}
ERRORS = {
    "PlayerStateError": PlayerStateError,
    "EmptyPlaylistError": EmptyPlaylistError,
    "ValueError": ValueError,
}
NONE = "—"


def read_examples(path: Path) -> list[dict[str, str]]:
    """Строки таблицы из раздела «Примеры» в виде словарей «колонка →
значение»."""
    lines = path.read_text(encoding="utf-8").split("## Примеры", 1)
[1].splitlines()
    rows = [
        [cell.strip() for cell in line.strip().strip("|").split("|")]
        for line in lines
        if line.startswith("|")
    ]
    header, _separator, *body = rows
    return [dict(zip(header, row)) for row in body]


EXAMPLES = read_examples(SPEC)


def apply(player: Player, action: str) -> None:
    name, *args = action.split()
    getattr(player, name)(*map(int, args))


@pytest.mark.parametrize("ex", EXAMPLES, ids=[f"N{ex['№']}:
{ex['Действия']}" for ex in EXAMPLES])
def test_playback_example(ex):
    player = Player()
    player.load(PLAYLISTS[ex["Плейлист"]])
    player.repeat = ex["Повтор"] == "да"
    *setup, last = [a.strip() for a in ex["Действия"].split(";")]

    for action in setup:
        apply(player, action)
    if ex["Ошибка"] == NONE:
        apply(player, last)
    else:
        with pytest.raises(ERRORS[ex["Ошибка"]]):
            apply(player, last)

    title = player.current_track.title if player.current_track else NONE
    assert title == ex["Трек"]
    assert player.state.value == ex["Состояние"]
    assert player.position == int(ex["Позиция"])
```

Приложение В. Сценарии BDD и определения шагов

Листинг В.1 – Сценарии features/playback.feature

```
# language: ru
Функция: Управление воспроизведением музыкального плеера
  Чтобы слушать музыку в удобном мне порядке
  Как пользователь музыкального плеера
  Я хочу запускать, приостанавливать и переключать треки

  Предыстория:
    Дано треки загружены в плеер:
      | название | исполнитель | длительность |
      | Intro    | Band        | 180          |
      | Song     | Band        | 200          |
      | Outro    | Band        | 150          |

  Сценарий: Запуск воспроизведения кнопкой «play»
    Когда я нажимаю "play"
    Тогда начинается воспроизведение
    И звучит трек "Intro"

  Сценарий: Пауза и продолжение воспроизведения
    Дано плеер воспроизводит трек "Song"
    Когда я нажимаю "pause"
    Тогда воспроизведение приостановлено
    И текущий трек "Song"
    Когда я нажимаю "play"
    Тогда начинается воспроизведение
    И звучит трек "Song"

  Сценарий: Переключение на следующий и предыдущий трек
    Дано плеер воспроизводит трек "Intro"
    Когда я нажимаю "next"
    Тогда звучит трек "Song"
    Когда я нажимаю "previous"
    Тогда звучит трек "Intro"

  Структура сценария: Переключение вперёд с последнего трека
    Дано плеер воспроизводит трек "Outro"
    И режим повтора <повтор>
    Когда я нажимаю "next"
    Тогда текущий трек "<трек>"
    И плеер в состоянии "<состояние>"

    Примеры:
      | повтор   | трек  | состояние       |
      | выключен | Outro | остановлен      |
      | включён  | Intro | воспроизведение |

  Структура сценария: Переключение назад с первого трека
    Дано плеер воспроизводит трек "Intro"
    И режим повтора <повтор>
    Когда я нажимаю "previous"
    Тогда текущий трек "<трек>"
    И плеер в состоянии "воспроизведение"

    Примеры:
      | повтор   | трек  |
      | выключен | Intro |
      | включён  | Outro |
```

Листинг В.2 – Сценарии features/empty_playlist.feature

```
# language: ru
Функция: Работа плеера без загруженных треков
  Чтобы понимать, почему музыка не играет
  Как пользователь музыкального плеера
  Я хочу получать понятное сообщение об ошибке

  Сценарий: Нажатие «play» при пустом плейлисте
    Дано плейлист пуст
    Когда я нажимаю "play"
    Тогда плеер сообщает об ошибке "Плейлист пуст"
    И плеер в состоянии "остановлен"
```

Листинг В.3 – Определения шагов features/steps/player_steps.py

```
from behave import given, step, then

from player.core import EmptyPlaylistError, PlayerState,
PlayerStateError, Track

STATES = {
    "воспроизведение": PlayerState.PLAYING,
    "пауза": PlayerState.PAUSED,
    "остановлен": PlayerState.STOPPED,
}
REPEAT = {"включён": True, "выключен": False}


@given("треки загружены в плеер:")
def step_load_tracks(context):
    context.player.load(
        Track(row["название"], row["исполнитель"],
int(row["длительность"]))
        for row in context.table
    )


@given("плейлист пуст")
def step_empty_playlist(context):
    context.player.load([])


@given('плеер воспроизводит трек "{title}"')
def step_playing_track(context, title):
    player = context.player
    player.play()
    for _ in player.playlist:
        if player.current_track.title == title:
            return
        player.next()
    raise AssertionError(f'Трек "{title}" отсутствует в плейлисте')


@given("режим повтора {mode}")
def step_repeat_mode(context, mode):
    context.player.repeat = REPEAT[mode]


@step('я нажимаю "{button}"')
def step_press(context, button):
```

```
044      try:
045          getattr(context.player, button)()
046      except (EmptyPlaylistError, PlayerStateError) as exc:
047          context.error = exc
048
049
050  @then("начинается воспроизведение")
051  def step_is_playing(context):
052      assert context.player.state is PlayerState.PLAYING,
context.player.state
053
054
055  @then("воспроизведение приостановлено")
056  def step_is_paused(context):
057      assert context.player.state is PlayerState.PAUSED,
context.player.state
058
059
060  @then('плеер в состоянии "{state}"')
061  def step_state(context, state):
062      assert context.player.state is STATES[state], context.player.state
063
064
065  @then('звучит трек "{title}"')
066  def step_track_sounds(context, title):
067      assert context.player.state is PlayerState.PLAYING,
context.player.state
068      assert context.player.current_track.title == title,
context.player.current_track
069
070
071  @then('текущий трек "{title}"')
072  def step_current_track(context, title):
073      assert context.player.current_track.title == title,
context.player.current_track
074
075
076  @then('плеер сообщает об ошибке "{message}"')
077  def step_error(context, message):
078      assert context.error is not None, "Ошибка не возникла"
079      assert str(context.error) == message, context.error
```

Листинг В.4 – Окружение behave features/environment.py

```
001  import sys
002  from pathlib import Path
003
004  sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
005
006  from player.core import Player  # noqa: E402
007
008
009  def before_scenario(context, scenario):
010      context.player = Player()
011      context.error = None
```

Приложение Г. Протокол итогового прогона тестов

Протокол получен командами python -m pytest -v --cov=player --cov-report=term-missing и python -m behave (файл logs/11_final_all.txt).

Листинг Г.1 – Протокол итогового прогона

```
$ python -m pytest -v --cov=player --cov-report=term-missing
============================= test session starts
==============================
platform linux -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0 --
/usr/bin/python3
hypothesis profile 'default'
rootdir: /home/fedorklochkov/university/practice3-music-player
configfile: pytest.ini
testpaths: tests
plugins: hypothesis-6.168.0, cov-7.1.0
collecting ... collected 50 items

tests/acceptance/test_player_api.py::test_ac1_load_playlist PASSED
[  2%]
tests/acceptance/test_player_api.py::test_ac2_play_starts_first_track PASSED
[  4%]
tests/acceptance/test_player_api.py::test_ac3_pause_and_resume PASSED
[  6%]
tests/acceptance/test_player_api.py::test_ac4_switch_tracks PASSED
[  8%]
tests/acceptance/test_player_api.py::test_ac5_end_of_playlist_stops PASSED
[ 10%]
tests/acceptance/test_player_api.py::test_ac6_play_with_empty_playlist PASSED
[ 12%]
tests/acceptance/test_player_api.py::test_ac7_pause_when_stopped PASSED
[ 14%]
tests/acceptance/
test_player_api.py::test_ac8_invalid_track_keeps_previous_playlist PASSED
[ 16%]
tests/spec/test_playback_spec.py::test_playback_example[N1: play] PASSED
[ 18%]
tests/spec/test_playback_spec.py::test_playback_example[N2: play; tick 60]
PASSED [ 20%]
tests/spec/test_playback_spec.py::test_playback_example[N3: play; tick 60;
pause; tick 30] PASSED [ 22%]
tests/spec/test_playback_spec.py::test_playback_example[N4: play; tick 60;
pause; play] PASSED [ 24%]
tests/spec/test_playback_spec.py::test_playback_example[N5: tick 60] PASSED [
26%]
tests/spec/test_playback_spec.py::test_playback_example[N6: play; tick 180]
PASSED [ 28%]
tests/spec/test_playback_spec.py::test_playback_example[N7: play; tick 190]
PASSED [ 30%]
tests/spec/test_playback_spec.py::test_playback_example[N8: play; tick 530]
PASSED [ 32%]
tests/spec/test_playback_spec.py::test_playback_example[N9: play; tick 540]
PASSED [ 34%]
tests/spec/test_playback_spec.py::test_playback_example[N10: play; tick 60;
next] PASSED [ 36%]
tests/spec/test_playback_spec.py::test_playback_example[N11: play; tick 60;
previous] PASSED [ 38%]
tests/spec/test_playback_spec.py::test_playback_example[N12: play; next; tick
3; previous] PASSED [ 40%]
```

```
tests/spec/test_playback_spec.py::test_playback_example[N13: play; next; tick
4; previous] PASSED [ 42%]
tests/spec/test_playback_spec.py::test_playback_example[N14: play; tick 60;
stop] PASSED [ 44%]
tests/spec/test_playback_spec.py::test_playback_example[N15: pause] PASSED
[ 46%]
tests/spec/test_playback_spec.py::test_playback_example[N16: play; tick -5]
PASSED [ 48%]
tests/spec/test_playback_spec.py::test_playback_example[N17: play] PASSED
[ 50%]
tests/unit/test_player.py::test_track_stores_metadata PASSED
[ 52%]
tests/unit/test_player.py::test_track_requires_title[empty] PASSED
[ 54%]
tests/unit/test_player.py::test_track_requires_title[spaces] PASSED
[ 56%]
tests/unit/test_player.py::test_track_requires_positive_duration[0] PASSED
[ 58%]
tests/unit/test_player.py::test_track_requires_positive_duration[-5] PASSED [
60%]
tests/unit/test_player.py::test_new_player_is_stopped_and_empty PASSED
[ 62%]
tests/unit/test_player.py::test_load_makes_first_track_current PASSED
[ 64%]
tests/unit/test_player.py::test_load_resets_playback PASSED
[ 66%]
tests/unit/test_player.py::test_load_copies_playlist PASSED
[ 68%]
tests/unit/test_player.py::test_play_starts_playback PASSED
[ 70%]
tests/unit/test_player.py::test_play_with_empty_playlist_raises PASSED
[ 72%]
tests/unit/test_player.py::test_play_twice_keeps_playing PASSED
[ 74%]
tests/unit/test_player.py::test_pause_while_playing PASSED
[ 76%]
tests/unit/test_player.py::test_pause_when_not_playing_raises[stopped] PASSED
[ 78%]
tests/unit/test_player.py::test_pause_when_not_playing_raises[paused] PASSED
[ 80%]
tests/unit/test_player.py::test_play_after_pause_resumes_same_track PASSED
[ 82%]
tests/unit/test_player.py::test_stop_stops_playback PASSED
[ 84%]
tests/unit/test_player.py::test_next_switches_to_following_track PASSED
[ 86%]
tests/unit/test_player.py::test_next_keeps_playing PASSED
[ 88%]
tests/unit/test_player.py::test_next_while_paused_stops PASSED
[ 90%]
tests/unit/test_player.py::test_next_on_last_track_stops_playback PASSED
[ 92%]
tests/unit/test_player.py::test_previous_switches_back PASSED
[ 94%]
tests/unit/test_player.py::test_previous_on_first_track_stays_on_first PASSED
[ 96%]
tests/unit/test_player.py::test_switching_with_empty_playlist_raises[next]
PASSED [ 98%]
tests/unit/
test_player.py::test_switching_with_empty_playlist_raises[previous] PASSED
[100%]

================================ tests coverage
```

```
================================
_________________ coverage: platform linux, python 3.14.7-final-0
_________________

Name                  Stmts   Miss  Cover   Missing
-----------------------------------------------------
player/__init__.py        0      0   100%
player/api.py            44      1    98%   37
player/core.py           95      1    99%   121
-----------------------------------------------------
TOTAL                   139      2    99%
============================== 50 passed in 0.34s
==============================


$ python -m behave
USING RUNNER: behave.runner:Runner
Функция: Работа плеера без загруженных треков #
features/empty_playlist.feature:2
  Чтобы понимать, почему музыка не играет
  Как пользователь музыкального плеера
  Я хочу получать понятное сообщение об ошибке
  Сценарий: Нажатие «play» при пустом плейлисте    #
features/empty_playlist.feature:7
    Дано плейлист пуст                              #
features/steps/player_steps.py:21
    Когда я нажимаю "play"                          #
features/steps/player_steps.py:42
    Тогда плеер сообщает об ошибке "Плейлист пуст" #
features/steps/player_steps.py:76
    И плеер в состоянии "остановлен"                #
features/steps/player_steps.py:60


Функция: Управление воспроизведением музыкального плеера #
features/playback.feature:2
  Чтобы слушать музыку в удобном мне порядке
  Как пользователь музыкального плеера
  Я хочу запускать, приостанавливать и переключать треки
  Функция: Управление воспроизведением музыкального плеера  #
features/playback.feature:2


  Сценарий: Запуск воспроизведения кнопкой «play»  #
features/playback.feature:14
    Дано треки загружены в плеер:                   #
features/steps/player_steps.py:13
      | название | исполнитель | длительность |
      | Intro    | Band        | 180          |
      | Song     | Band        | 200          |
      | Outro    | Band        | 150          |
    Когда я нажимаю "play"                          #
features/steps/player_steps.py:42
    Тогда начинается воспроизведение                #
features/steps/player_steps.py:50
    И звучит трек "Intro"                           #
features/steps/player_steps.py:65


  Сценарий: Пауза и продолжение воспроизведения  #
features/playback.feature:19
    Дано треки загружены в плеер:                  #
features/steps/player_steps.py:13
      | название | исполнитель | длительность |
      | Intro    | Band        | 180          |
      | Song     | Band        | 200          |
```

```
      | Outro    | Band        | 150          |
    Дано плеер воспроизводит трек "Song"         #
features/steps/player_steps.py:26
    Когда я нажимаю "pause"                      #
features/steps/player_steps.py:42
    Тогда воспроизведение приостановлено         #
features/steps/player_steps.py:55
    И текущий трек "Song"                        #
features/steps/player_steps.py:71
    Когда я нажимаю "play"                       #
features/steps/player_steps.py:42
    Тогда начинается воспроизведение             #
features/steps/player_steps.py:50
    И звучит трек "Song"                         #
features/steps/player_steps.py:65


  Сценарий: Переключение на следующий и предыдущий трек  #
features/playback.feature:28
    Дано треки загружены в плеер:                #
features/steps/player_steps.py:13
      | название | исполнитель | длительность |
      | Intro    | Band        | 180          |
      | Song     | Band        | 200          |
      | Outro    | Band        | 150          |
    Дано плеер воспроизводит трек "Intro"        #
features/steps/player_steps.py:26
    Когда я нажимаю "next"                       #
features/steps/player_steps.py:42
    Тогда звучит трек "Song"                     #
features/steps/player_steps.py:65
    Когда я нажимаю "previous"                   #
features/steps/player_steps.py:42
    Тогда звучит трек "Intro"                    #
features/steps/player_steps.py:65


  Структура сценария: Переключение вперёд с последнего трека -- @1.1   #
features/playback.feature:44
    Дано треки загружены в плеер:                                      #
features/steps/player_steps.py:13
      | название | исполнитель | длительность |
      | Intro    | Band        | 180          |
      | Song     | Band        | 200          |
      | Outro    | Band        | 150          |
    Дано плеер воспроизводит трек "Outro"                              #
features/steps/player_steps.py:26
    И режим повтора выключен                                           #
features/steps/player_steps.py:37
    Когда я нажимаю "next"                                             #
features/steps/player_steps.py:42
    Тогда текущий трек "Outro"                                         #
features/steps/player_steps.py:71
    И плеер в состоянии "остановлен"                                   #
features/steps/player_steps.py:60


  Структура сценария: Переключение вперёд с последнего трека -- @1.2   #
features/playback.feature:45
    Дано треки загружены в плеер:                                      #
features/steps/player_steps.py:13
      | название | исполнитель | длительность |
      | Intro    | Band        | 180          |
      | Song     | Band        | 200          |
      | Outro    | Band        | 150          |
```

```
    Дано плеер воспроизводит трек "Outro"                                  #
features/steps/player_steps.py:26
    И режим повтора включён                                                #
features/steps/player_steps.py:37
    Когда я нажимаю "next"                                                 #
features/steps/player_steps.py:42
    Тогда текущий трек "Intro"                                             #
features/steps/player_steps.py:71
    И плеер в состоянии "воспроизведение"                                  #
features/steps/player_steps.py:60

  Структура сценария: Переключение назад с первого трека -- @1.1   #
features/playback.feature:56
    Дано треки загружены в плеер:                                          #
features/steps/player_steps.py:13
      | название | исполнитель | длительность |
      | Intro    | Band        | 180          |
      | Song     | Band        | 200          |
      | Outro    | Band        | 150          |
    Дано плеер воспроизводит трек "Intro"                                  #
features/steps/player_steps.py:26
    И режим повтора выключен                                               #
features/steps/player_steps.py:37
    Когда я нажимаю "previous"                                             #
features/steps/player_steps.py:42
    Тогда текущий трек "Intro"                                             #
features/steps/player_steps.py:71
    И плеер в состоянии "воспроизведение"                                  #
features/steps/player_steps.py:60

  Структура сценария: Переключение назад с первого трека -- @1.2   #
features/playback.feature:57
    Дано треки загружены в плеер:                                          #
features/steps/player_steps.py:13
      | название | исполнитель | длительность |
      | Intro    | Band        | 180          |
      | Song     | Band        | 200          |
      | Outro    | Band        | 150          |
    Дано плеер воспроизводит трек "Intro"                                  #
features/steps/player_steps.py:26
    И режим повтора включён                                                #
features/steps/player_steps.py:37
    Когда я нажимаю "previous"                                             #
features/steps/player_steps.py:42
    Тогда текущий трек "Outro"                                             #
features/steps/player_steps.py:71
    И плеер в состоянии "воспроизведение"                                  #
features/steps/player_steps.py:60

2 features passed, 0 failed, 0 skipped
8 scenarios passed, 0 failed, 0 skipped
46 steps passed, 0 failed, 0 skipped
Took 0min 0.001s
```